

Arquitetura de Computadores

Resumo de estudo — Conceitos fundamentais, processador, registradores e interrupções

■ **Objetivo do resumo** — Apresentar os conceitos básicos de um computador, as arquiteturas Von Neumann e Harvard, a distinção CISC vs. RISC, a organização interna do processador (UC, ULA, registradores) e o mecanismo de interrupções.

Seção 1 — Conceitos Básicos

1. O que é um Computador?

■ **Computador** — Máquina genérica de processamento de dados. Recebe dados de entrada, processa-os e produz informação (saída).



As quatro funções fundamentais de um computador genérico são: **armazenar, processar, transformar e controlar** dados.

2. Arquiteturas de Computador

Arquitetura	Característica principal
Von Neumann	Dados e instruções compartilham a mesma memória e o mesmo barramento. Mais simples, mas pode gerar gargalo (bottleneck de Von Neumann).
Harvard	Memórias separadas para dados e instruções, com barramentos independentes. Permite busca simultânea de instrução e dado (usada em microcontroladores).

3. Microcontroladores

Um microcontrolador integra em um único chip: CPU, memória principal, controladores e interfaces de E/S, todos interligados por barramentos internos.

Seção 2 — Arquitetura de Processadores (CISC vs. RISC)

Aspecto	CISC	RISC
Conjunto de instruções	Grande quantidade, com múltiplos modos de endereçamento	Poucas instruções, todas de mesma largura (formato fixo)
Microcódigo	Usa microprogramação: microcódigo reside em memória de controle (ROM dentro da UC, mais rápida que RAM)	Não requer microcódigo, sobrando mais espaço no chip para outros recursos

Aspecto	CISC	RISC
Novas instruções	Basta ter espaço na memória de controle (custo baixo para adicionar)	Exige redesenho do hardware (instruções fixas em silício)
Pipelining	Difícil: instruções de tamanhos e durações variáveis	Uso intenso: formato fixo facilita o paralelismo
Acesso à memória	Qualquer instrução pode acessar a memória diretamente	Apenas LOAD e STORE acessam memória (arquitetura de registradores)
Execução	Instruções complexas podem levar múltiplos ciclos	Execução rápida: idealmente 1 instrução por ciclo

■ **Contexto histórico** — A filosofia RISC nasceu ligada ao Unix e às máquinas de terminal. A ideia era simplificar o hardware e deixar o compilador fazer o trabalho pesado de otimização.

Seção 3 — O Processador (CPU)

1. Função

■ **Processador (CPU)** — Responsável pela execução de programas armazenados na memória principal. Seu ciclo básico é: **buscar** instruções, **examinar** (decodificar) e **executar**.

2. Componentes Básicos da CPU

Componente	Sigla	Função
Unidade de Controle	UC	Movimentação de dados e instruções de/para o processador. Conecta-se a todos os elementos da CPU. Emite sinais de controle em intervalos fixos, originados por um gerador de sinais (clock), medido em GHz.
Unidade Lógica e Aritmética	ULA	Realiza operações lógicas (AND, OR, NOT...) e aritméticas (+, -, *, /). Pode utilizar um ou dois valores de entrada.
Registradores	Regs	Pequenas memórias de alta velocidade dentro da CPU. Armazenam resultados temporários e informações de controle. Podem ser de uso geral ou de uso específico.
Barramentos internos	—	Vias de intercomunicação entre os componentes da CPU.

■ **SRAM vs. DRAM** — Os registradores são memórias do tipo **SRAM** (Static RAM): muito rápidas, mas caras e de baixa densidade. Já a memória principal é do tipo **DRAM** (Dynamic RAM), comumente chamada apenas de RAM: mais lenta, mas barata e de alta capacidade.

3. Registradores Especiais

Registrador	Sigla	Função
Program Counter	PC	Indica o endereço da próxima instrução a ser buscada para execução.
Instruction Register	IR	Mantém a instrução que está sendo executada no momento.
Memory Address Register	MAR	Armazena o endereço de memória que será acessado na próxima operação.
Memory Buffer Register	MBR	Contém o dado lido da memória ou o dado a ser escrito nela.
Stack Pointer	SP	Aponta para o topo da pilha (stack) na memória.
Program Status Word	PSW	Armazena flags de estado do processador (carry, zero, overflow, sinal, etc.).

Seção 4 — Interrupções

■ **Interrupção** — Mecanismo que interrompe o ciclo normal de instrução do processador, desviando a execução para um tratador específico. Após o tratamento, o processador retoma a execução de onde parou.

1. Causas de Interrupção

Tipo	Origem	Exemplos
Programa	Gerada pela própria execução do programa	Overflow aritmético; divisão por zero; instrução ilegal
Timer	Temporizador interno do processador	Preempção de processos; contagem de tempo do SO
E/S	Controlador de Entrada/Saída	Disco terminou leitura; teclado pressionado; pacote de rede chegou
Hardware	Falha de componente físico	Erro de paridade de memória; falha de barramento

Síntese — Mapa do Resumo

Seção	Tema Central	Conceito-chave
1	Conceitos Básicos	Computador = armazenar, processar, transformar, controlar dados
2	CISC vs. RISC	CISC = muitas instruções complexas; RISC = poucas, rápidas, pipeline

Seção	Tema Central	Conceito-chave
3	Processador (CPU)	UC + ULA + Registradores + Barramentos
4	Interrupções	Programa, Timer, E/S, Hardware

Resumo Rápido — Cola de Estudo

■ **Checklist** — • Definir computador e suas 4 funções fundamentais. • Diferenciar Von Neumann e Harvard (memória compartilhada vs. separada). • Comparar CISC e RISC em pelo menos 4 aspectos. • Explicar o papel da microprogramação no CISC. • Descrever o ciclo básico do processador (buscar, decodificar, executar). • Listar e explicar os componentes da CPU (UC, ULA, registradores). • Distinguir SRAM (registradores) de DRAM (memória principal). • Identificar a função de cada registrador especial (PC, IR, MAR, MBR, SP, PSW). • Definir interrupção e classificar por tipo (programa, timer, E/S, hardware).

■ **Regras de ouro** — 1) Von Neumann = simplicidade, mas gargalo no barramento compartilhado. 2) RISC simplifica o hardware e confia no compilador para otimizar. 3) O clock (GHz) define o ritmo da UC — mais rápido = mais instruções por segundo. 4) Interrupções são essenciais para multitarefa: sem elas, o SO não conseguiria alternar entre processos.

■ **Conexão com outras áreas** — Sistemas Operacionais dependem diretamente do mecanismo de interrupções para escalonamento de processos e gerência de E/S. Compiladores precisam conhecer a arquitetura (CISC/RISC) para gerar código eficiente. Redes neurais em hardware (aceleradores como GPUs e TPUs) exploram massivamente o paralelismo herdado da filosofia RISC.

#arquitetura #computadores #cpu #cisc #risc #registradores #interrupcoes